HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 09

PR Process Automation, Quality Gates and LLM-as-Judge

Back to engineering: turning what this cohort already believes about a good PR into an automated, evidence-based quality gate.

DURATION
1.5 Hours · Module 9 of 12

FORMAT
Guided Configure + Judge-a-PR Lab

DELIVERED FOR
Honeywell Embedded Engineering

NIIT StackRoute × RPS Programme Design

How We'll Spend These Ninety Minutes

Six connected moves — from why this matters to a flawed PR your strategy has to catch.

01

Why This Module Exists

10 min

What this cohort already believes makes a PR approvable — in their own words.

02

What a Quality Gate Checks

15 min

Spec compliance, standards, test evidence, build results, regression & safety.

03

Rubric Design

15 min

Turning “what needs the closest check” into scored evaluation criteria.

04

LLM-as-Judge Workflow

15 min

Evidence gathering, rubric scoring, confidence, and the escalation branch.

05

Configure & Judge a Flawed PR

25 min

Run the full quality flow against a deliberately broken embedded change.

06

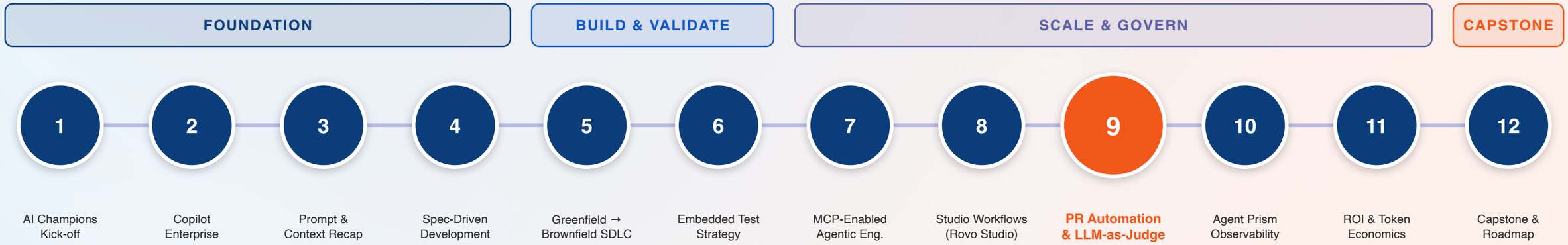
Review-Cycle Debrief

10 min

Compare review time and quality — automated gate vs. manual-only.

Module 09 in the 12-Module Journey

Back to engineering — where every discipline from Modules 04–06 gets checked automatically, every time.



Why this matters: This is the traceability chain from Module 04, the test evidence from Module 06, and the tool-calling discipline from Module 07 — all converging into a single automated checkpoint before merge.

Why This Module Exists, In This Cohort's Own Words

Module 01 showed 18 of 19 declined a PR that merely passed tests. Here's what they actually wrote.

> *“Passing tests and meeting the specification are necessary but not sufficient conditions for approval.”*

> *“It has to be optimized for memory and be consistent with implementation patterns and design.”*

> *“An AI reviewer should either request changes or flag the PR for human review.”*

> *“If AI makes a mistake, humans need to find the bug — so it has to be understandable by humans.”*

This module's job: turn these already-sound instincts into a rubric an LLM-as-Judge can apply consistently, every single time — not just when a careful reviewer happens to notice.

What a PR Quality Gate Checks

Six categories, one automated pass — before a human ever opens the diff.

Specification Compliance

Every acceptance criterion from Module 04 either satisfied, or flagged as unmet.

Coding & Design Standards

House style, naming conventions & architectural patterns — checked, not assumed.

Test Evidence

Module 06's traceability table & pass/fail output, attached and verifiable.

Compiler / Build Results

Clean build across the full target toolchain — not just “it compiled once.”

Regression Risk

Full existing suite run, not just new tests — Module 06's regression discipline.

Security & Safety Considerations

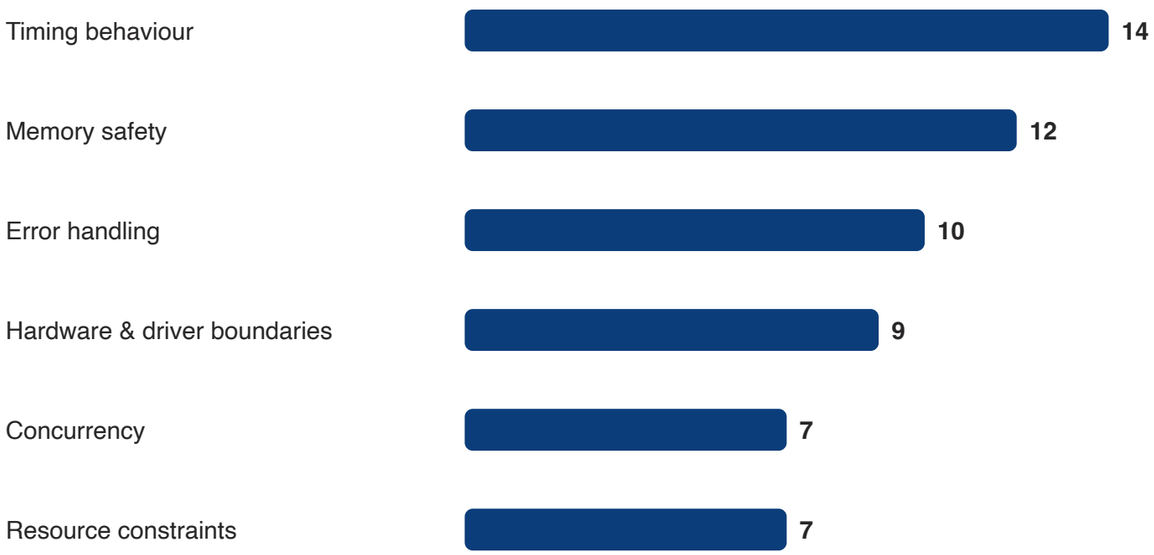
Memory safety, resource limits & safety-critical patterns — checked explicitly.

Nothing new here: every category above is a discipline from an earlier module — this gate just checks that the discipline actually happened.

Rubric Design: What Needs the Closest Check

The same calibration data from Module 01 — now turned into scored evaluation criteria.

When AI Writes Embedded C/C++, What Needs Closest Human Check?



HOW THE RUBRIC WEIGHTS THIS

Timing & memory

Automatic fail if violated — no confidence threshold saves it.

Error handling & boundaries

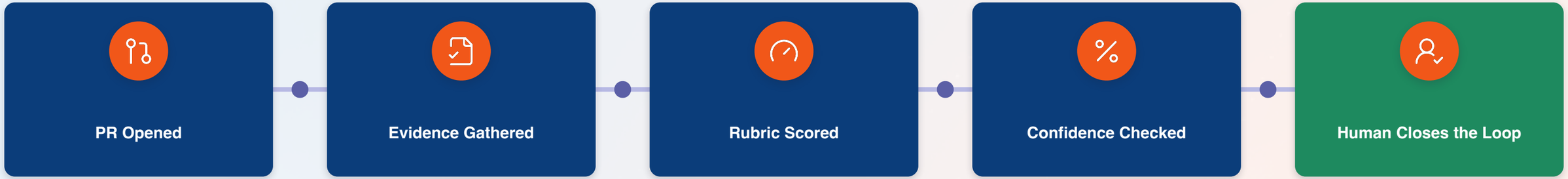
Heavily weighted; flagged for human review if uncertain.

Concurrency & resource use

Scored, but more often escalated than auto-failed.

LLM-as-Judge: How Automated Review Actually Works

Five stages, one branch point — confidence decides whether a human sees it first.



Change lands against the spec from Module 04, ready for review.

Test results, build logs & static analysis pulled automatically — Module 06's package.

Each category from Slide 6 scored against the evidence, not a guess.

High confidence → comment posted. Low confidence → escalate.

Approval always stays a human decision — the judge informs, never merges.

The name is precise: “judge,” not “approver” — it evaluates and informs; the merge decision is never delegated away.

Evidence-Based Comments vs. Vague Feedback

The same PR, reviewed two ways — only one gives the author something to act on.

VAGUE FEEDBACK

"This looks risky, might cause issues later. Consider reviewing the memory usage before merging."

- ✕ No number: how much memory, how measured?
- ✕ "Might cause issues" — no specific failure mode named
- ✕ No reference to the design pattern being ignored
- ✕ Author has nothing concrete to act on

EVIDENCE-BASED COMMENT

"Heap usage +18% vs. baseline (build log, line 42). Diverges from the pool-allocator pattern used in can_driver.c. AC-3 requires <2KB overhead — not met. Flagging for human review."

- ✓ Exact figure, cited against a build artifact
- ✓ Names the specific pattern & file being diverged from
- ✓ Ties directly back to a Module 04 acceptance criterion

Notice: the evidence-based version reads like the calibration survey's own quotes — specific, cited, and pointed at exactly what needs to change.

Security & Safety Considerations in PR Review

The category that never gets a confidence-based pass — flagged is flagged, full stop.

Memory Safety

Buffer bounds, use-after-free & allocation patterns — checked against the sanitizer output from Module 06.

Resource Constraints

Stack, heap & timing budgets verified against the platform's actual limits, not a rule of thumb.

Hardware & Driver Boundaries

Changes touching HAL code get an automatic escalation — no exceptions, regardless of confidence score.

Safety-Critical Patterns

Interrupt handling, state-machine integrity & fail-safe behaviour — named explicitly in the rubric.

The one hard rule in this whole module: a high confidence score never overrides a security or safety flag — it only ever adds a human, never removes one.

Human Escalation: When the Judge Isn't Sure

Low confidence isn't a failure of the system — it's the system working as designed.

HIGH CONFIDENCE → AUTO-COMMENT

- Evidence is complete and unambiguous — test results, build logs & static analysis all agree.
- Rubric score is clear-cut against the specification's stated thresholds.
- The judge posts an evidence-based comment directly, exactly like Slide 8's example.

LOW CONFIDENCE → ESCALATE

- Evidence is incomplete, conflicting, or the change touches security/safety territory.
- Rubric score sits in ambiguous ground — not a clean pass, not a clean fail.
- The judge attaches everything it found and routes to a named human reviewer — Module 07's escalation path, applied here.

Calibrate the threshold, don't eliminate it: a judge that never escalates isn't confident — it's miscalibrated.

Review-Cycle Reduction: The Business Case

What today's lab measures — the same four signals from Module 03, applied to review instead of implementation.

Review Time

Wall-clock time from PR opened to merge-ready — automated gate vs. manual-only.

Review-Cycle Count

How many back-and-forth rounds it took to reach an approvable state.

Issues Caught Pre-Human

What the automated gate found before a reviewer ever opened the diff.

Escalation Rate

Share of PRs routed to a human — the number that should feel right, not zero.

Where this goes next: these numbers feed the token/cost and ROI model in Module 11 — review-cycle reduction is one of its clearest inputs.

Hands-On Lab: Configure, Flag, Judge, Close the Loop

Four stages, one deliberately flawed PR — the same defect-injection discipline from Module 06.

Configure

Set up a PR quality flow with the rubric from Slide 6, wired to a real repository.

Flag

Submit a deliberately flawed embedded change — memory regression, standards violation, or both.

Judge

Run AI review & LLM-as-Judge; capture evidence from tests, build & static analysis.

Close the Loop

Compare review time & quality against a manual pass, then close with human approval.

Success looks like: the gate catches your planted defect and explains why — in language as specific as the calibration survey's own quotes.

Anti-Patterns: How PR Automation Goes Wrong

Five habits that turn a quality gate into rubber-stamping with extra steps.

DO THIS	NOT THAT
Cite specific evidence: line numbers, measurements, build artifacts	Post vague comments like “this looks risky, please review”
Escalate automatically on security & safety, regardless of confidence	Let a high overall score suppress a safety-relevant flag
Treat a low-confidence escalation as the system working	Tune the judge to escalate less often just to feel more “autonomous”
Keep merge as a human decision, every single time	Let a consistently high-scoring judge auto-merge “just this once”
Feed the judge the same evidence a human reviewer would want	Let it score based on assumptions the evidence doesn't support

Module 09 Outcomes & What's Next

Everything below should be true before we monitor all of this in Agent Prism, Module 10.

- Can name what a PR quality gate checks: spec, standards, tests, build, regression, safety
- Can trace the LLM-as-Judge workflow: evidence → score → confidence → branch
- Can write an evidence-based comment, not a vague one
- Configured a real PR quality flow and watched it catch a planted defect
- Built a rubric from real evidence — what the cohort already flags as needing closest check
- Know why security & safety flags never yield to a high confidence score
- Understand the escalation path and why a healthy escalation rate isn't zero
- Know the five anti-patterns that turn a quality gate into rubber-stamping

NEXT — MODULE 10: AGENT PRISM FOR MONITORING, OBSERVABILITY AND ENTERPRISE CONTROL

2 hours · Traces, failure patterns, token/cost leakage & adoption tracking — connecting everything built so far to measurable outcomes.

Questions & Discussion

Module 09 — PR Process Automation, Quality Gates and LLM-as-Judge